# OOP Concepts – Constructor, Method, Function

Topic 5 - functions

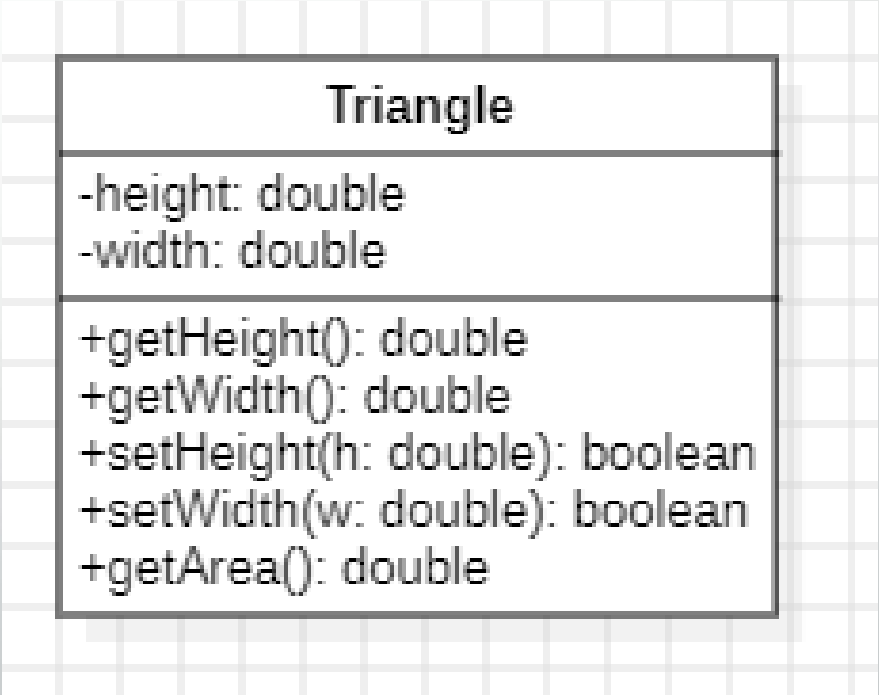
# Intended Learning Outcomes

After completing the lesson, students should be able to:

- Create class with properties, methods and constructors
- Distinguish between Pass-by-value and Pass-by-reference
- Apply method overloading for constructor and methods
- Write Javadoc documentation
- Generate API documentation using javadoc

# Recap – UML Class diagram

- Triangle class
  - A class to encapsulate width and height
  - Getters and Setters
    - Setters – only set field if positive parameter.
    - Return false if error.
  - Avoid stale data – getArea() calculates
    - Area = ½ width * height



# Importance of Data Hiding/Encapsulation

- Avoid Interference between code
  - E.g. Triangle and Rectangle used width and height
- Easier maintenance of code
  - E.g. check before store the value
- Code Re-use
  - Inheritance (later)

```
# Triangle.py
width = 3.0
height = 4.0
area = 1/2.0 * width * height

# Another Rectangle
width = 4.0
height = 3.0
area = width * height

print("Area of triangle is " + area)
```

# Triangle.java

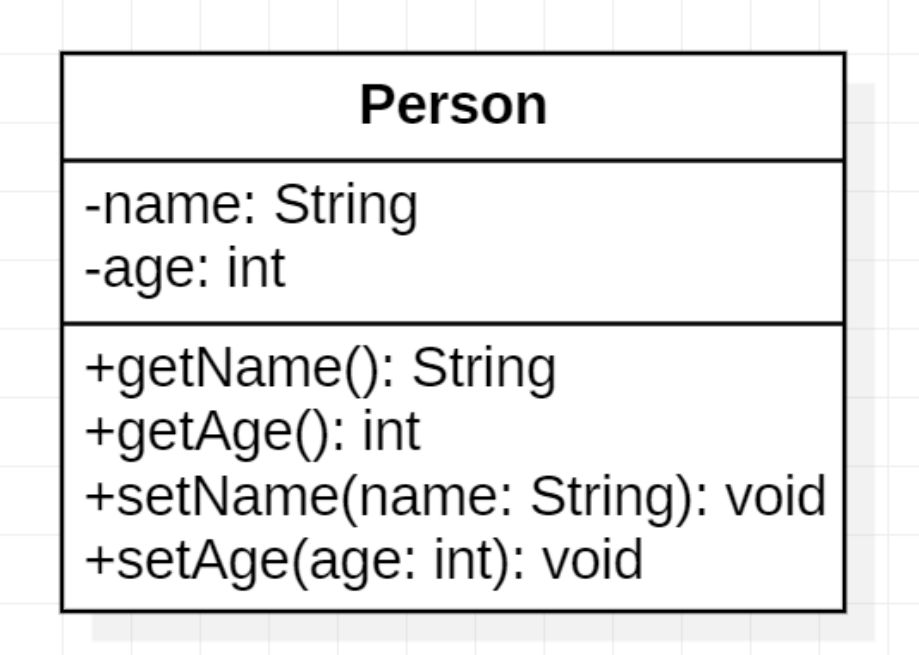
```
public class Triangle{
    // fields
    private double height;
    private double width;
    // getters
    public double getHeight(){ return height;}
    public double getWidth(){ return width;}
    // setters
    public boolean setHeight(double h){
        if (h>0){ this.height = h; return true;
        }else{ return false;}
    }
    public boolean setWidth(double w){
        if (w>0){ width = w; return true; }
        else return false;
    }
    public double getArea(){
        return 1/2.0 * width * height;
    }
} // class Triangle
```

# main() method of Triangle.java

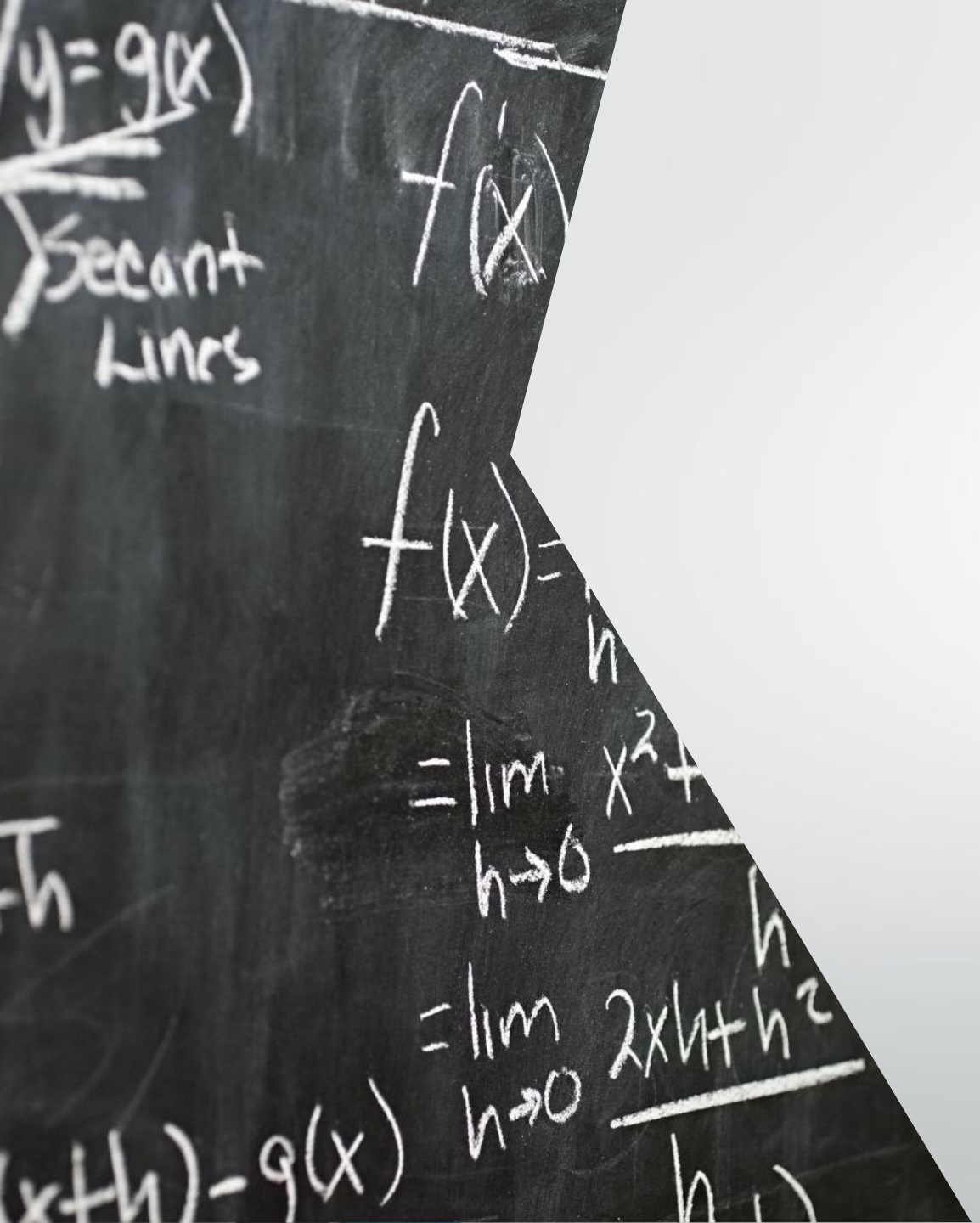
```
// main method for testing
    public static void main(String[] args) {
        Triangle t = new Triangle();
        t.setHeight(3);
        t.setWidth(4);
        System.out.println(
            "Area of a triangle with width (" + t.getWidth() + ") and " +
            "height (" + t.getHeight() +") is "
            + t.getArea() + ".");

    } // main()
```

# Exercise - Person class

- Create a Plain Old Java Object (POJO) class
- Person has 2 private attributes
- name is a String
- age is an integer
- There are relevant getter & setter without checking




https://en.wikipedia.org/wiki/Plain_old_Java_object




# Instance Methods

# Usage of methods

- Simplify programs (code reuse)
  - Group reusable code together
  - Write once, execute multiple times
- Divide & conquer
  - Break problem into smaller pieces

# Method Syntax

- a block of code that performs a specific task.
- encapsulated within a class
- can be called from other parts
- Method Header
  - Access modifier
  - Return Type
  - Method Name
  - Parameter List
- Method Body
  - Statements
  - Return statement (if applicable)

Access modifier  Method Name

```
public static void main(String[] args){
  // method body
    return;
}
```

Return Type  Parameter List

# Method Syntax

- `Access Modifier`: specifies the visibility / accessibility
  - can be `public`, `private`, `protected`, or
  - can be omitted (default package-private access).
- `Return Type`: data type of the return value
  - Primitive (e.g. int, double) or
  - Class type (e.g. String, Object)
  - void: does not return any value
- `Method Name`: used to identify and call the method.
- `Parameter List`: comma-separated list of input parameters
  - parameter consists of data type & parameter name
- Method Body
  - block of code enclosed within curly braces `{}`
  - contains the statements
- Return Statement
  - If the method has a return type other than `void`,
    - must include a return statement
    - to return a value of the specified type.

# Method Example

Publicly accessible

Call by this name

Accept 2 integer parameters

```
public int calculateSum(int num1, int num2) {
        int sum = num1 + num2;
        return sum;
    }
```

Return integer

# Invoke / Call a method

```
MethodSyntax m = new MethodSyntax();
int result = m.calculateSum(10, x);
```

- For object methods
  - Call by ObjectName.MethodName(parameters)
  - Need to create an instance before calling the method
- main() method
  - Automatically called for Java application

# Passing Arguments

- When providing value / reference to a method
  - Data type must be compatible
  - Widening: automatic
  - Narrowing: Explicit type casting needed
- Parameter passing
  - By value - primitive types
  - By reference - Object

# Pass-by-value

- all primitive data types (such as int, float, boolean) are passed by value.
- When a method is called with a primitive parameter,
  - a copy of the value is made and passed to the method.
- Any changes made to the parameter within the method
  - **do not** affect the original value outside the method.

# Pass-by-value example

```java
public class PassByValueExample {
    public void changeValue(int num) {
        num = 10; // Changes the value of the local 'num' variable
    }
    public static void main(String[] args) {
        PassByValueExample obj = new PassByValueExample();
        int value = 5;
        obj.changeValue(value);
        System.out.println(value);
        // Output: 5 (original value remains unchanged)
    }
} // class
```

# Pass-by-reference

- objects (including arrays) are passed by reference.
- When an object is passed as a parameter to a method,
  - a copy of the reference to the object is made,
  - not a copy of the actual object.
- Changes made to the object within the method
  - **will affect** the original object outside the method.
- Reference itself is passed by value.
  - reassign the reference within the method,
  - **not affect** the original reference outside the method.

# PassByRefExample.java

```
public class PassByRefExample {
    public void changeValue(Integer num) {
        num = 10; // Changes the value of the local 'num' variable
    }
    public static void main(String[] args) {
        PassByValueExample obj = new PassByValueExample();
        Integer value = 5;
        obj.changeValue(value);
        System.out.println(value); // Output: 5 (value changed in the
Reference)
    }
} // class
```

# Wrapper Classes

- Objects that encapsulate primitive data types,
  - providing additional methods and functionalities to work with the values.
- Purpose
  - Enable use of primitive data types as objects.
  - Provide utility methods for converting, comparing, and manipulating values.
  - Facilitate working with collections (e.g., ArrayLists) that require objects.
- Key Features: Auto-boxing, Unboxing

| Primitive Data Type | Wrapper Class |
| --- | --- |
| byte | Byte |
| short | Short |
| int | Integer |
| long | Long |
| float | Float |
| double | Double |
| char | Character |
| boolean | Boolean |

Reference: https://www.w3schools.com/java/java_wrapper_classes.asp

# String as immutable objects

- `String` class is immutable,
  - once a `String` object is created,
  - its value cannot be changed.
- also applies when `String` objects are passed as parameters to methods.
- When a `String` is passed as a parameter to a method,
  - unable to modify the contents of the `String` object itself.

# PassStringExample.java

```java
public class PassStringExample {
    public void changeString(String str) {
        str = str.toLowerCase();
        str = "World"; // Does not modify the original String object
    }
    public static void main(String[] args) {
        PassStringExample obj = new PassStringExample();

        String greeting = "Hello";
        obj.changeString(greeting);
        System.out.println(greeting);
        // Output: Hello (original String remains unchanged)
    }// main
} // class
```

# Constructors

# Need of “Constructor”

- When an instance (object) of a class is created
  - Need to initialize instance fields
  - Initialization tasks
- Java default constructor
  - Provided automatically
  - Same name as class name, but no return value
    - E.g.: Triangle()

# Java Constructors

- a special method that is used to initialize objects of a class.
- called when an object is created using the `new` keyword.
- Properties
  - Same name as the class name
  - No return type (no 'void' as well)
  - Mostly public
    - Unless internal use (private)
  - Default constructor with no argument is provided
    - Initialize all fields using default value

# Default constructor

- Behavior of default constructor
  - Java provides a constructor with no parameters
    - Not provided if there are constructors.
  - Sets numeric fields to `0 (or 0.0)`
  - Boolean fields to false
  - Object reference variable to null
    - A value to indicate 'nothing' assigned

# Constructor Examples (Person.java)

```java
// Constructor with parameters
public Person(String name, int age) {
    this.name = name;
    this.age = age;
}

// Constructor without parameters (default constructor)
public Person() {
    this("Unknown", 18); // Call the other constructor
}
```

Constructor call must be the first statement in a constructor

# Using Constructor Example (Person.java)

```
public static void main(String[] args) {
    Person p1 = new Person(); // Default Constructor
    Person p2 = new Person("Alex", 21); // Constructor with arguments
    System.out.println(p1.getName() + " age: " + p1.getAge());
    System.out.println(p2.getName() + " age: " + p2.getAge());
} // main
```

# Full specific constructor

- a constructor that accepts parameters for **all** the instance variables of a class.
- provide values for all the attributes of an object during its creation
- Example

```
Person(String name, int age)
```

# Constructor using Builder Pattern

- Builder Pattern:
  - A creational design pattern aim to simplify the creation of complex objects.
  - Ideal for objects with many optional parameters.
- Advantages of Builder Pattern
  - Improves Readability: Clear and readable code.
  - Immutable Objects: Ensures immutability.
  - Flexible Construction: Allows step-by-step construction.

# Implementation steps

- Create a Static Nested Builder Class: Define a static nested class inside the main class.
- Add Fields to the Builder Class: Include all fields from the main class.
- Add Methods to Set Values: Provide methods to set values for each field.
- Add a Build Method: Return an instance of the main class.
- Private Constructor in Main Class:  Accept a builder object as a parameter.

# Example: PersonBuilder.java

```java
// nested Builder class
public class PersonBuilder {
    private String name;
    private int age;

    public PersonBuilder() {
        // Default constructor
    }

    public PersonBuilder name(String name) {
        this.name = name;
        return this;
    }

    public PersonBuilder age(int age) {
        this.age = age;
        return this;
    }

    public Person build() {
        return new Person(name, age);
    }
}
```

# Attribute Scope

Class, Instance and Local Variable

# Scoping

- refers to the context in which variables and attributes are defined and can be accessed.
- helps to determine the visibility and lifetime of variables and attributes,
- can exist at different levels within a program.
  - Global Scope - accessible in the whole program (Not in Java)
  - Class Scope
  - Instance Scope
  - Local Scope

```java
public class Square {
    static private int NumSquare = 0;
    private int length=0;
    public Square(int len){
        int local_length = len;

        NumSquare++;
        this.length = len;
        System.out.println(local_length);
    }
    public String toString(){
        return "Square(" + this.length + ")";
    }
    // Other methods ... (skip)
    public static void main(String[] args){
        Square sq1 = new Square(5);
        Square sq2 = new Square(7);
        System.out.println(Square.NumSquare);
        System.out.println(sq1.toString());
        System.out.println(sq2.length);
    }
}
```

# Class Scope (Static Attributes):

- Static attributes belong to the class itself and
- shared across all instances of the class.
- Accessible using the class name.
  - e.g. `Square.NumSquare`

# Instance Scope (Instance Attributes):

- Instance attributes belong to individual objects created from a class.
- Each instance has its own copy of the attributes.
- Accessible using the object instance.
  - e.g. `sq2.length`

# Local Scope (Local Variables):

- Variables declared within a method or block of code.
- Only accessible within that method or block.
- Exist temporarily during the execution of the method.

# Special Class - Enumeration

# Enum (Enumeration)

- A special data type in Java that represents a fixed set of constants.
  - used to define a collection of named constants.
- Purpose
  - Provide a clear, type-safe way to represent fixed sets of related values.
  - Improve code readability and maintainability.
  - Prevent invalid values from being assigned to variables.

# Key points of `enum`

- Enums are defined using the `enum` keyword.
- Enum constants are implicitly `public`, `static`, and `final`.
- Enums can have fields, methods, and constructors.
- Enum constants can be used in switch statements.

# EnumExample.java

```
enum Weekdays {
    SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY
}
public class EnumExample {
    public static void main(String[] args) {
        Weekdays today = Weekdays.WEDNESDAY;
        System.out.println("Today is: " + today);
    }
}
```

# JavaDoc

# JavaDoc

- Method documentation
  - Comments before definition
- Javadoc comments
  - Begin with `/**`
  - End with `*/`
- Regular comments
  - `// for single line`
  - `/* multiple lines comment */`

```
// JavaDocExample.java
/**
 * [Description of the method]
 *
 * @param parameter1 Description of parameter1
 * @param parameter2 Description of parameter2
 * @return Description of the return value
 * @throws ExceptionType Description of the
exception(s) thrown
 */
public returnType methodName(parameter1,
parameter2) throws ExceptionType {
    // Method implementation
}
```

# Javadoc Example

- @param parameter1 Description of parameter1
- @return Description of the return value
- @throws ExceptionType Description of the exception(s) thrown

```
/**
 * Calculates the sum of two numbers.
 *
 * @param num1 The first number.
 * @param num2 The second number.
 * @return The sum of num1 and num2.
 */
    public int calculateSum(int num1, int num2)
{
        int sum = num1 + num2;
        return sum;
    }
```

# Javadoc - documentation generation tool

- a powerful tool
- simplifies the process of generating API documentation
- making it easier to understand, use, and maintain Java projects.
- Trigger in Command / Terminal:

```
javadoc -d docs *.java
```

- -d for output directory (e.g. docs)

# Exercise - Student

- Write a Java class of Student with the following features
  - Attributes: name (String), year (int), gpa (double)
  - Constructors:
    - Full Specific Constructor,
    - Default constructor: name: "", year: 1, gpa: 0
  - Methods:
    - relevant getter and setters
    - print(): Print the student record with "name (year: year#) [GPA: gpa]" format
- Documentation - provide Javadoc documentation for the class
- main() method
  - 1 create 2 students record using different constructors.
  - 2. Call their print() method to print the record.